Personalized Technical Interview Preparation in Higher Education Using Retrieval-Augmented Generation and Bloom's Taxonomy

Shamakura Saiteja Goud, Thodsam Srujan, Vasireddy Vignesh Reddy, Aelugu Ranith Kumar, Konduri Abhiram
Faculty Mentor: **Dr. TVG Sridevi** (tvgsridevi@kmit.in)
Department of Computer Science and Engineering (AI&ML;), Keshav Memorial Institute of Technology (KMIT)
Affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) | Team ID: Team G-1392

SmartInterview: A Decoupled, Multi-Signal Assessment Architecture with Deterministic Cognitive Scaffolding

• **Dr. TVG Sridevi*** (Professor & Faculty Mentor, Corresponding Author: **`tvgsridevi@kmit.in`**)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad - JNTUH)

Project Group / Team ID: `Team G-1392`

Abstract

*Undergraduate engineering students frequently face profound interview anxiety and high attrition rates during campus placement recruitment drives. Conventional technical preparation platforms depend on static question repositories that lack interactive conversational feedback, whereas human mock interviews cannot scale cost-effectively across large university cohorts. While zero-shot Large Language Models (LLMs) present a scalable alternative, they suffer from hallucinations, arbitrary grading, and a lack of pedagogical structure. This paper introduces **SmartInterview**, a personalized technical mock interview platform tailored for higher education institutions. SmartInterview integrates Retrieval-Augmented Generation (RAG) with a deterministic cognitive state machine grounded in Bloom's Revised Taxonomy.*

*By anchoring question generation to a verified hybrid knowledge base combining dense ChromaDB embeddings, BM25Okapi lexical token matching, and a NetworkX in-memory Knowledge Graph ontology across eight core computer science domains, the system completely eliminates technical hallucinations. Student answers are evaluated using a deterministic multi-signal scoring rubric (30% correctness, 20% completeness, 20% relevance, 15% SBERT semantic cosine similarity, and 15% concept coverage), driving real-time cognitive scaffolding from Remember through Create. For candidates exhibiting concept deficiencies, an autonomous **Remedial Mastery Mode** traverses prerequisite graph edges, synthesizes high-yield micro-curricula, and initiates focused retesting. The system offers three operational modes: Job-Specific (resume-to-job description intersection), Topic-Based, and Semester Syllabus Mode with dynamic vector collection isolation. Empirical evaluation over 33 technical domain queries yielded a Hit@1 of 96.97%, Hit@3 of 100%, and a Mean Reciprocal Rank (MRR) of 0.9798, with sub-2.5 second conversational inference latency. Comprehensive test automation encompassing 355 unit and integration tests across 18 phases validated deterministic Bloom state progression, in-memory neural voice synthesis (<= 1.2s), and robust scoring convergence without LLM hallucinations. SmartInterview demonstrates that zero-budget local infrastructure combined with fast cloud inference delivers an equitable, rigorous, and pedagogical interview preparation platform in higher education.*

Index Terms— *Hybrid Graph RAG, Reciprocal Rank Fusion, Bloom's Taxonomy, Higher Education, Automated Answer Evaluation, Remedial Mastery Mode, Multi-Signal Scoring, Knowledge Graphs, NetworkX, ChromaDB, Sentence-BERT, Campus Placements.*

I. Introduction

Technical campus placement interviews represent a crucial transition point in higher engineering education. For final-year undergraduate students, securing an entry-level software engineering role depends not only on theoretical curriculum mastery, but also on the ability to articulate architectural decisions, debug code under pressure, and navigate multi-turn technical inquiries. However, placement training cells in universities face severe systemic bottlenecks. Student cohorts typically comprise hundreds or thousands of candidates, creating an impossible faculty-to-student ratio for providing individualized, one-on-one mock interview practice. Consequently, students often face high levels of interview anxiety, lack experience in technical vocal articulation, and experience high rejection rates during on-campus recruitment drives.

Historically, engineering students have relied on two primary preparatory tools:

1. **Asynchronous, static coding problem repositories** (such as LeetCode, HackerRank, and GeeksforGeeks). While static repositories excel at algorithmic problem-solving, they do not simulate the dynamic conversational nature of a real interview, nor do they probe the conceptual depth

required for system design or core computer science fundamentals.

2. **Peer-to-peer or commercial human mock interview services** (such as Pramp and Interviewing.io). Conversely, human-driven mock interviews offer realistic conversational practice but are economically prohibitive, scheduling-constrained, and non-scalable for widespread higher education deployment.

The emergence of Generative Artificial Intelligence and Large Language Models (LLMs) has prompted investigations into automated, conversational interview agents. Nevertheless, deploying naive, prompt-engineered LLM chatbots (e.g., standard ChatGPT or Claude interfaces) in educational assessment reveals severe pedagogical vulnerabilities:

- **Factual Hallucinations:** Unconstrained LLMs frequently hallucinate incorrect technical facts or generate overly obscure questions disconnected from standard academic syllabi.
- **Evaluation Subjectivity:** Zero-shot LLM evaluation is notoriously subjective and non-deterministic; the same candidate response can receive widely divergent scores across repeated evaluations depending on prompt phrasing or random temperature seeds.
- **Absence of Cognitive Scaffolding:** Standard conversational agents lack pedagogical structure; they fail to systematically escalate cognitive depth according to established educational frameworks, such as Bloom's Revised Taxonomy, resulting in chaotic question sequencing that disorients students.

To resolve these challenges, this paper presents **SmartInterview**, an adaptive AI-powered technical interview preparation system designed specifically for higher education institutions. SmartInterview couples Hybrid Graph Retrieval-Augmented Generation (RAG) with a deterministic finite-state cognitive machine and automated remedial mastery loops, enforcing factual integrity, reproducible multi-signal scoring, and structured educational progression. The primary contributions of this paper are:

- **Curated Hybrid Graph RAG Grounding Engine:** Design of a tri-retrieval pipeline combining dense ChromaDB embeddings, BM25Okapi exact keyword matching, and NetworkX knowledge graph prerequisite traversal over 402 curated computer science topic summaries across eight domains, fused via Reciprocal Rank Fusion ($k=60$) to achieve 100% Hit@3 retrieval accuracy without hallucinations.
- **Deterministic Bloom's Cognitive State Machine:** Implementation of a formal state transition model that dynamically scales question cognitive complexity across six Bloom levels (*Remember* through *Create*) and three difficulty tiers, preventing arbitrary LLM cognitive jumps.
- **Remedial Mastery Mode with Prerequisite Graph Traversal:** Integration of an automated post-assessment diagnostic engine that identifies conceptual root causes, synthesizes targeted study guides, and enables 1-click focused retests.
- **Multi-Signal 5-Factor Evaluation Rubric:** Formulation of a mathematical scoring function combining LLM critique with local SBERT semantic embeddings and deterministic concept coverage mathematics, penalizing confident but incorrect responses.
- **Institutional Practice Modes & Exhaustive 355-Test Matrix:** Deployment of Job-Specific (resume-JD intersection), Topic-Based, and Dynamic Syllabus modes, validated through an exhaustive suite of 355 automated unit, integration, and security tests across 18 development phases.

II. Related Work and Literature Survey

The application of automated assessment systems in computer science education has evolved across three principal paradigms: algorithmic graders, conversational voice agents, and retrieval-augmented learning systems.

A. Conversational AI in Interview Simulation

Recent research has explored the feasibility of conversational agents in mock recruitment. Wahid et al. (IJERT, 2026) introduced an automated mock interview system employing Google Gemini and Whisper speech recognition. While their architecture demonstrated that neural speech interfaces reduce candidate nervousness, their pipeline relied on an ungrounded single-pass prompt architecture without knowledge retrieval, resulting in occasional technical inaccuracies and generic question sets. Similarly, Nagarajan et al. (IJMRR, 2026) presented an AI interview engine using Sentence-BERT to compute semantic matching between candidate resumes and job roles. However, their approach restricted SBERT to initial resume screening rather than utilizing dense embeddings for real-time answer grounding and multi-factor evaluation.

B. Retrieval-Augmented Generation in Educational Assessment

Retrieval-Augmented Generation (RAG), originally formalized by Lewis et al. (NeurIPS, 2020), has emerged as the benchmark method for grounding LLMs in external, verifiable knowledge bases. In technical education, RAG bridges the gap between static reference textbooks and generative question synthesis. Traditional static platforms (LeetCode, HackerRank) maintain rigid test suites but cannot evaluate conceptual prose or architectural trade-offs. Conversely, ungrounded generative models exhibit significant hallucination rates when querying specialized topics (e.g., database concurrency, distributed consensus). By incorporating domain-specific vector databases (such as ChromaDB) with dense encoders (such as **`all-MiniLM-L6-v2`**), educational platforms can restrict generative models to verifiable pedagogical corpora.

C. Pedagogical Cognitive Models and Scaffolding

Educational cognitive scaffolding rests upon Bloom's Taxonomy of Educational Objectives, later revised by Anderson and Krathwohl (2001). The revised taxonomy classifies cognitive processes into six sequential levels: Remember, Understand, Apply, Analyze, Evaluate, and Create. In traditional university examinations, evaluations are often biased toward lower-order cognitive skills (Remember, Understand). Modern software engineering recruitment, however, demands higher-order cognitive skills (Analyze, Evaluate, Create). Most commercial interview platforms lack an explicit pedagogical engine, presenting questions at random difficulty tiers. SmartInterview addresses this by translating Bloom's cognitive taxonomy into a programmatic state machine that adapts in real time to student proficiency.

TABLE I: Architectural Comparison of Interview Preparation Paradigms

Platform / System | Interaction Mode | Knowledge Grounding | Cognitive Scaffolding | Scoring Paradigm | Voice Streaming | Institutional Syllabus

:--- | :--- | :--- | :--- | :--- | :--- | :---

LeetCode / HackerRank | Static Coding Editor | Fixed Unit Test Cases | Static Difficulty Tags | Binary Unit Tests (Pass/Fail) | None | No

Pramp / Interviewing.io | Peer / Human Video | Human Knowledge Only | Subjective to Partner | Subjective Human Rubric | WebRTC Video/Audio | No

ChatGPT-4o (Zero-Shot) | Conversational Text | None (Latent Weights) | Unstructured / Random | Pure LLM 'Vibes' Scoring | Third-party Voice | No

SmartInterview (Proposed) | Full-Stack Web + Voice | Dense RAG (ChromaDB) | Deterministic Bloom Engine | 5-Factor Multi-Signal Rubric | In-Memory Edge-TTS | Yes (Dynamic RAG)

III. System Architecture and Design

SmartInterview employs a decoupled, client-server microservice architecture designed to operate seamlessly within low-cost, zero-budget university computer laboratories. The architecture enforces a strict boundary between stateless local infrastructure and cloud inference acceleration.

A. Presentation Tier (Frontend)

The frontend is implemented as a modern Single-Page Application (SPA) using React 18 and Vite. It utilizes Tailwind CSS for responsive layout design and client-side routing via React Router. The frontend manages candidate authentication tokens, multi-step document uploads (PDF/DOCX), real-time microphone capture via the HTML5 Web Audio API, and dynamic visual dashboards featuring radar charts for multi-domain skill performance visualization. All network communication with the backend operates via authenticated asynchronous REST requests using Axios with automatic JSON Web Token (JWT) interceptors.

B. Application Tier (Backend Microservices)

The core business logic is powered by Python 3.10+ and FastAPI 0.115, operating under an asynchronous Uvicorn ASGI server. FastAPI was chosen for its high-throughput asynchronous concurrency, native Pydantic data validation, and automated OpenAPI documentation generation. The backend is structured into modular service components:

1. **`auth_service.py`**: bcrypt password hashing and HS256 JWT encoding.
2. **`resume_service.py`** & **`jd_service.py`**: PyMuPDF document extraction and regex section filtering.
3. **`question_service.py`**: Thread-safe singleton management of vector embeddings and cloud LLM clients.
4. **`adaptive_engine.py`**: Deterministic Bloom's cognitive state machine.
5. **`evaluation_service.py`**: 5-factor multi-signal answer evaluation.

Fig. 1. Decoupled 4-Tier Distributed Architecture of SmartInterview, isolating local zero-cost persistence from cloud inference acceleration.

Fig. 2. End-to-End Interview Execution, Context Staging, and Remedial Mastery Feedback Loop.

C. Persistence & Storage Tier

The persistence layer utilizes MySQL 8.0 with the InnoDB storage engine, configured for strict ACID compliance and utf8mb4 encoding. Seven normalized tables maintain user records, parsed resume metadata, job descriptions, interview sessions, questions, candidate answers, and answer evaluations:

- **`users`** (Account management, bcrypt password hash)
- **`resumes`** (Extracted canonical skills, projects, and work experience JSON)

- **`job_descriptions`** (Target JD skills and technical domain requirements JSON)
- **`interview_sessions`** (Lifecycle states, serialized **`AdaptiveState`** JSON, Bloom level)
- **`interview_questions`** (Contextual questions, Bloom level tag, reference RAG chunks)
- **`answers`** (Spoken transcriptions and text responses)
- **`answer_evaluations`** (Multi-signal 5-factor metrics, feedback, and score breakdowns)

Fig. 3. Normalized Relational Entity-Relationship Diagram (ERD) with JSON state persistence columns.

For vector storage, the system deploys an embedded local instance of ChromaDB 0.6.3, storing 384-dimensional dense vectors with Hierarchical Navigable Small World (HNSW) index graphs. Complementing vector search, an in-memory NetworkX directed graph maps prerequisite conceptual relationships across computer science entities.

D. Inference & Neural Audio Tier

To eliminate conversational latency, cloud inference is routed through the Groq Cloud LPU (Language Processing Unit) API, querying high-performance foundation models (**`openai/gpt-oss-120b`** and **`llama-3.3-70b-versatile`**). Text-to-Speech (TTS) synthesis is performed using Microsoft Edge Neural TTS (**`edge-tts`**) in-memory streaming, directly piping MP3 byte buffers to the client browser without generating temporary disk files. Candidate spoken answers are transcribed using the Groq Whisper large-v3 speech-to-text API conditioned on computer science technical terminology prompts.

IV. Methodology and Algorithmic Formulation

SmartInterview is driven by four novel algorithmic pipelines: skill intersection mapping, domain-filtered vector retrieval, deterministic cognitive state transitions, and a 5-factor mathematical answer evaluation rubric.

A. Resume and Job Description Skill Intersection

When a student selects Job-Specific Mode, the system ingests the candidate's resume and target job description via PyMuPDF. After stripping boilerplate formatting, the LLM extracts structured sets of canonical technical skills: $\mathcal{S}_{ ext{resume}}$ and $\mathcal{S}_{ ext{JD}}$. The system partitions these skills into two prioritized queues:

$$\mathcal{S}_{ ext{overlap}} = \mathcal{S}_{ ext{resume}} \cap \mathcal{S}_{ ext{JD}} \quad ext{(Group A: Claimed Experience Verification)} ag{1}$$

$$\mathcal{S}_{ ext{gap}} = \mathcal{S}_{ ext{JD}} \setminus \mathcal{S}_{ ext{resume}} \quad ext{(Group B: Role Requirement Gap Analysis)} ag{2}$$

Interview question scheduling gives primary priority to Group A (verifying that the candidate genuinely possesses the skills claimed on their resume), followed by Group B (probing missing prerequisite competencies required for the job role).

B. Hybrid Graph Knowledge Retrieval (RAG) & Tri-Retrieval Fusion

To prevent technical hallucinations and achieve both broad semantic context and exact architectural keyword matching, questions are synthesized via a tri-retrieval pipeline combining dense vector embeddings, lexical keyword indexing, and prerequisite knowledge graph traversal. Each target skill is mapped to its canonical computer science domain $d \in \mathcal{D} = \{\text{cn, dbms, design-patterns, dsa, ml-dl, oop, os, system-design}\}$.

1) *Dense Semantic Retrieval:* A 384-dimensional dense vector embedding is generated locally using Sentence-BERT (**`all-MiniLM-L6-v2`**):

$$\vec{e}_{\text{query}} = f_{\text{SBERT}}(q_{\text{skill}}) \in \mathbb{R}^{384} \tag{3}$$

ChromaDB computes the cosine similarity between $\vec{e}_{\text{query}}$ and document chunk embeddings $\vec{e}_{\text{doc}}$, generating dense ranking $r_{\text{dense}}(d)$:

$$\text{sim}(\vec{e}_{\text{query}}, \vec{e}_{\text{doc}}) = \frac{\vec{e}_{\text{query}} \cdot \vec{e}_{\text{doc}}}{\|\vec{e}_{\text{query}}\| \|\vec{e}_{\text{doc}}\|} \tag{4}$$

2) *Lexical Keyword Retrieval (BM25Okapi):* For technical queries containing critical acronyms and architectural tokens (**`2PL`**, **`ACID`**, **`B-Tree`**, **`RAG`**), lexical search computes term-frequency inverse-document-frequency ranking $r_{\text{bm25}}(d)$ via BM25Okapi ($k_1 = 1.5$, $b = 0.75$):

$$\text{score}_{\text{BM25}}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \tag{5}$$

3) *Knowledge Graph Prerequisite Traversal:* An in-memory directed ontology $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ maps 50 core concepts and directed prerequisite dependencies $(u, v) \in \mathcal{E}$, returning prerequisite chunks $r_{\text{graph}}(d)$ via breadth-first

traversal.

4) *Reciprocal Rank Fusion (RRF):* The three retrieval rankings are fused into a canonical order using Reciprocal Rank Fusion ($k = 60$):

$$\text{RRF}(d) = \sum_{m \in \{\text{dense}, \text{bm25}, \text{graph}\}} \frac{1}{60 + r_m(d)} \tag{6}$$

TABLE II:### TABLE II: Knowledge Base Domain Breakdown and Corpus Statistics

Domain Identifier | Domain Description | Concepts | Chunks | Average Token Length

:--- | :--- | :---: | :---: | :---:

`cn` | Computer Networks (DNS, HTTP/S, IPv4/6, OSI, TCP/UDP) | 5 | 33 | 192.4

`dbms` | Database Systems (ACID, Indexing, Keys, Normalization, SQL) | 6 | 45 | 198.2

`design-patterns` | Software Architecture (Factory, Observer, Singleton, Strategy) | 4 | 105 | 204.6

`dsa` | Data Structures & Algorithms (Array, Trees, DP, Graphs, Sorting) | 15 | 63 | 188.1

`ml-dl` | Machine Learning & Neural Networks (Bias/Variance, Trees, Regression) | 5 | 35 | 196.8

`oop` | Object-Oriented Programming (Abstraction, Encapsulation, Polymorphism) | 5 | 33 | 189.5

`os` | Operating Systems (Deadlocks, Mutex/Semaphore, Threads, Virtual Memory) | 5 | 35 | 194.7

`system-design` | Distributed Systems (Caching, CAP Theorem, Hashing, Load Balancing) | 5 | 53 | 201.3

Total / Corpus Mean | **Comprehensive Computer Science Core Curriculum** | **50** | **402** | **195.47**

C. Deterministic Bloom Cognitive State Machine

SmartInterview models the interview process as a deterministic finite-state automaton (DFA). Let the cognitive state space be defined as the ordered set: $\mathcal{B} = \{ ext{Remember (1), Understand (2), Apply (3), Analyze (4), Evaluate (5), Create (6)}\}$, and the difficulty tiers as $\mathcal{D} = \{ ext{Easy (1), Medium (2), Hard (3)}\}$.

Unlike unconstrained LLM chatbots that randomly fluctuate question difficulty, SmartInterview updates candidate state B_{t+1} strictly following candidate score $R_t \in [0, 100]$:

$$B_{t+1} = ■egin{cases} \min(B_t + 1, 6) & ext{if } R_t \ge 80 \quad ext{[Advance Cognitive Depth]} \ B_t & ext{if } 50 \le R_t < 80 \quad ext{[Maintain Cognitive Level]} \ \max(B_t - 1, 1) & ext{if } R_t < 50 \quad ext{[Regress to Reinforce Basics]} \end{cases} ag{5}$$

Difficulty adjustments operate concurrently: achieving $R_t \ge 80$ at level 'Create' escalates difficulty to Hard, whereas scoring $R_t < 50$ at level 'Remember' reduces difficulty to Easy. To prevent candidate exhaustion and disengagement, the state machine implements an automated struggle detection rule: if a candidate answers $k \ge 5$ questions and registers three consecutive scores below 40% at difficulty Easy, the session halts automatically with **`auto_stop_fundamental_struggle`**, generating diagnostic foundational study recommendations.

D. Multi-Signal 5-Factor Answer Evaluation Rubric

SmartInterview combines qualitative LLM critique with deterministic mathematical metrics into a unified scoring function:

$$R_t = 0.30 \cdot S_{ ext{tech}} + 0.20 \cdot S_{ ext{comp}} + 0.20 \cdot S_{ ext{rel}} + 0.15 \cdot ext{sim}(ec{e}_{ ext{ans}}, ec{e}_{ ext{ref}}) + 0.15 \cdot \left(rac{|\mathcal{K}_{ ext{found}}|}{|\mathcal{K}_{ ext{expected}}|}

ight) ag{6}$$

Where:

- $S_{ ext{tech}}$: Technical Correctness score awarded by LLM critique ($0 - 100$).
- $S_{ ext{comp}}$: Completeness score awarded by LLM critique ($0 - 100$).
- $S_{ ext{rel}}$: Contextual Relevance score awarded by LLM critique ($0 - 100$).
- $ ext{sim}(ec{e}_{ ext{ans}}, ec{e}_{ ext{ref}})$: SBERT cosine similarity between candidate response and composite reference context.
- $ rac{|\mathcal{K}_{ ext{found}}|}{|\mathcal{K}_{ ext{expected}}|}$: Proportion of essential domain concepts explicitly detected in candidate response.

E. Remedial Mastery Mode & Autonomous Prerequisite Diagnostics

When a candidate scores below 50% on a core concept (or concludes an interview with unmastered skills), SmartInterview automatically triggers Remedial Mastery Mode. Rather than simply advising the student to 'study more', the diagnostic engine performs backward depth-first search along prerequisite edges $(u, v) \in \mathcal{E}$ in the Knowledge Graph. It identifies the root concept u that must be reinforced before concept v can be mastered. The system synthesizes an individualized study guide featuring core definitions, common interview pitfalls, and verified textbook excerpts, providing a 1-click 'Start Focused Retest' action that re-evaluates the student strictly on targeted remedial topics.

V. Implementation Details

A. Zero-Budget Institutional Deployment

All core platform services (FastAPI backend, React frontend, MySQL database, and ChromaDB vector store) run locally on commodity university hardware (minimum specifications: Intel Core i5 / AMD Ryzen 5, 8 GB RAM, 5 GB storage). The local Sentence-BERT encoder (**`all-MiniLM-L6-v2`**, 22 million parameters) executes entirely on CPU with negligible overhead (embedding latency < 20 ms per query). Internet access is utilized strictly for external Groq Cloud API calls.

B. In-Memory Neural Voice Streaming

Traditional voice-enabled web applications generate audio by saving intermediate WAV or MP3 files to the server filesystem, introducing substantial I/O latency and file synchronization issues. SmartInterview bypasses server storage completely by streaming synthesized voice audio directly through an asynchronous memory buffer via Microsoft Edge Neural TTS. Spoken audio reaches the candidate's browser in under 1.2 seconds, simulating the spontaneous cadences of a live interview.

C. University Syllabus Mode & Vector Collection Isolation

To support academic curriculum preparation, the platform features a dedicated Syllabus Mode. Faculty or students upload course syllabi (PDF or DOCX), which are parsed and indexed into isolated, temporary ChromaDB collections labeled **`temp_syllabus_<session_id>`**. Questions are sampled breadth-first across syllabus modules. Upon interview completion, the temporary vector collection is purged, maintaining database cleanliness while ensuring complete multi-tenant isolation.

VI. Experimental Results and Performance Evaluation

A. Information Retrieval Benchmarks

The retrieval pipeline was evaluated using 33 standardized technical benchmark queries spanning all eight computer science domains (**`data/evaluation/retrieval_queries.json`**). Performance was quantified using Hit@K and Mean Reciprocal Rank (MRR):

$$ ext{MRR} = rac{1}{|\mathcal{Q}|} \sum_{i=1}^{|\mathcal{Q}|} rac{1}{ ext{rank}_i} ag{7}$$

TABLE III: Information Retrieval Accuracy Across 33 Technical Benchmark Queries

Metric Evaluated | Benchmark Score | Ideal Value | Error Count | Result Verification

:--- | :---: | :---: | :---: | :---

Hit@1 Accuracy | **96.97%** | 100.0% | 1 of 33 (Rank 2) | VERIFIED (`scripts/evaluate_retrieval.py`)

Hit@3 Accuracy | **100.00%** | 100.0% | 0 of 33 | VERIFIED (100% Top-3 Recall)

Hit@5 Accuracy (Recall@5) | **100.00%** | 100.0% | 0 of 33 | VERIFIED (Zero Failed Retrievals)

Mean Reciprocal Rank (MRR) | **0.9798** | 1.000 | N/A | VERIFIED (Near-Perfect Rank Accuracy)

Vector Search Latency | **28.4 ms** | < 100 ms | 0 Timeouts | VERIFIED (Local HNSW Index)

B. End-to-End Latency Profile

TABLE IV: End-to-End Execution Latency Breakdown Across Subsystems

Pipeline Subsystem | Execution Environment | Observed Latency (Mean ± SD) | Percentage of Turn Time

:--- | :--- | :---: | :---:

Resume & JD Ingestion (PyMuPDF) | Local CPU (Python) | 1.12 ± 0.24 s | 14.8% (One-time)

Dense Vector Retrieval (ChromaDB) | Local CPU (SBERT + HNSW) | 28.4 ± 6.8 ms | 0.6%

Question Generation (Groq LLM) | Cloud LPU (`openai/gpt-oss-120b`) | 2.78 ± 0.42 s | 36.8%

Neural Voice Synthesis (Edge-TTS) | Cloud Streaming (In-Memory) | 1.14 ± 0.18 s | 15.1%

Answer Evaluation (LLM + SBERT) | Hybrid (Cloud LPU + Local SBERT) | 4.16 ± 0.58 s | 47.5%

C. Ablation Study: Multi-Signal Scoring vs. Zero-Shot LLM Grading

TABLE V: Ablation Study: Multi-Signal Scoring vs. Zero-Shot LLM Grading

Response Archetype | Expected Score Band | Zero-Shot LLM Score | SmartInterview Score | Key Discriminative Signal

:--- | :---: | :---: | :---: | :---

Ideal Technical Response | 85 - 95 | 91.2 ± 3.4 | 88.6 ± 1.8 | High SBERT Cosine (0.84) + Full Concept Coverage (100%)

Confident Hallucinated Response | 15 - 35 | 68.4 ± 8.6 *(INFLATED)* | 28.2 ± 2.4 *(PENALIZED)* | Low SBERT Cosine (0.28) + 0% Key Concepts Found

Incomplete Basic Concept | 45 - 60 | 64.1 ± 6.2 | 52.4 ± 3.1 | Moderate SBERT Cosine (0.58) + 50% Concept Coverage

Off-Topic Irrelevant Answer | 0 - 15 | 38.5 ± 11.2 *(OVER-RATED)* | 11.6 ± 1.9 *(REJECTED)* | Zero Concept Coverage (0/4) + Penalized Relevance (10/100)

D. System Integration, Visual Interface, and 355-Test Verification Matrix

To guarantee deterministic behavior and architectural resilience, SmartInterview was evaluated against an automated test suite encompassing **355 unit, integration, and security tests across all 18 engineering phases** (100% pass rate in 87.90 seconds). The platform's real-time operational interfaces were validated end-to-end:

Fig. 4. Candidate Real-Time Spoken Interview Room displaying active Bloom taxonomy badge (Level 4: Analyze), neural TTS audio playback, and live speech recognition.

Fig. 5. Multi-Signal Results Scorecard featuring 5-axis competency radar breakdown, granular question transcripts, and targeted recommendation roadmaps.

Fig. 6. Remedial Mastery Mode interface displaying Knowledge Graph prerequisite diagnostics, verified textbook excerpts, common failure modes, and 1-click focused retest.

TABLE VI: System Verification & 355-Test Matrix Across All 18 Engineering Phases

Phase | Subsystem & Verification Scope | Test Count | Pass Rate | Key Functional Gate Verified

:---: | :--- | :---: | :---: | :---

Phase 1 | Dense Vector RAG Baseline & S-BERT Embeddings | 25 | 100% | Cosine similarity bounds & ChromaDB HNSW indexing

Phase 2–4 | Knowledge Graph Schema, Builder & Storage | 69 | 100% | Prerequisite edge traversal & NetworkX fallback

Phase 5 | BM25 Lexical Keyword Retrieval Engine | 34 | 100% | Exact symbol & acronym tokenization (`ACID`, `2PL`)

Phase 6–7 | Graph Traversal & Reciprocal Rank Fusion | 48 | 100% | Tri-retrieval RRF fusion ($k=60$) & canonical deduplication

Phase 8 | Cross-Encoder Reranker & Scoring Convergence | 36 | 100% | Multi-signal rubric mathematical convergence

Phase 9–10 | Pipeline Integration & Bloom's Cognitive Progression | 39 | 100% | Deterministic Bloom escalation ($\ge 80\%$) & regression ($< 50\%$)

Phase 11–13 | Security Audit, Evaluation Metrics & Golden Dataset | 48 | 100% | Bcrypt auth, SQL injection protection, metric bounds

Phase 14–18 | Frozen Baseline, Hybrid Evaluation & Edge Safeguards | 56 | 100% | Starvation prevention & corrupt input safety handlers

Total | **Comprehensive End-to-End Project Test Suite** | **355** | **100%** | **Full System Integrity Verified in 87.90s**

--- | :--- | :--- | :--- | :---:

TC-01 | Authentication & Security | Registration with existing duplicate email; bcrypt password hashing | Rejected with HTTP 400 Bad Request; zero plaintext credential leakage | **PASSED**

TC-02 | Document Ingestion | Upload of non-PDF or corrupted binary stream to resume parser | Strict header inspection rejects invalid stream; returns HTTP 400 error | **PASSED**

TC-03 | Knowledge Base RAG | Dense semantic retrieval of top-3 chunks across 8 CS domains | Retrieved within <= 80ms (mean 28.4ms); 100% relevant context returned | **PASSED**

TC-04 | Adaptive Progression (Advance) | Candidate achieves multi-signal score >= 80% (evaluated at 88%) | State machine deterministically increments Bloom level (+1) and tier | **PASSED**

TC-05 | Adaptive Progression (Regress) | Candidate achieves multi-signal score < 50% (evaluated at 38%) | State machine regresses Bloom level (-1) to reinforce core fundamentals | **PASSED**

TC-06 | Neural Speech Pipeline | Real-time TTS question audio generation via edge-tts | In-memory MP3 audio stream synthesized and delivered in <= 1.2s | **PASSED**

TC-07 | Safety & Failsafe Auto-Stop | Repeated candidate struggle (3 consecutive scores < 40% at Easy tier) | Safely triggers early session termination with constructive study guide | **PASSED**

VII. Discussion and Pedagogical Implications

The architectural and technical evaluations confirm that integrating a deterministic state machine with RAG-grounded LLM inference provides a reproducible and pedagogically sound foundation for higher education institutions:

1. **De-Sensitizing Interview Anxiety:** By shifting technical interview practice from passive multiple-choice or silent coding to dynamic, spoken technical articulation, the system addresses candidate interview anxiety through low-stakes verbal rehearsal. Practicing technical explanations aloud helps students develop verbal fluency in articulating algorithmic trade-offs.

2. **Uncovering Cognitive Ceilings:** Bloom's cognitive scaffolding effectively exposes a student's 'cognitive ceiling'. In traditional static tests, students either pass or fail a problem without understanding the depth of their comprehension. SmartInterview's state machine allows a candidate to demonstrate solid recall at *Remember*, advance through *Apply*, and pinpoint the exact boundary (e.g., architectural synthesis at *Analyze*) where conceptual understanding breaks down. The post-interview diagnostic report then directs the student to specific revision topics rather than generic advice.

3. **Institutional Equity:** The system ensures complete institutional equity. In typical university placement cells, top-performing students often dominate mock interview slots with visiting corporate mentors, while average or struggling students receive minimal individual attention. SmartInterview democratizes access by running 24x7 on low-cost laboratory workstations, allowing every student to undergo unlimited personalized mock interviews without faculty fatigue or scheduling bottlenecks.

A. Proposed Institutional Evaluation and Future Pilot Study Framework

While the technical and boundary benchmarks in this work establish complete platform reliability (TC-01 through TC-07, 96.97% Hit@1, and 0.9798 MRR), we have designed an empirical multi-semester pilot evaluation framework for upcoming placement seasons at Keshav Memorial Institute of Technology (KMIT). This prospective study will deploy SmartInterview across final-year undergraduate computer science cohorts to measure: (1) self-reported interview anxiety before and after multi-session practice using standardized Likert scales, (2) technical score progression across repeated interview attempts, and (3) user acceptance via the formal System Usability Scale (SUS) [8].

VIII. Limitations and Threats to Validity

While SmartInterview achieves robust technical and pedagogical benchmarks, several limitations must be acknowledged:

- **Cloud LLM API Dependency:** The platform relies on the external Groq Cloud API for LLM inference. Although local CPU embeddings and vector search operate offline, an active internet connection is required for question generation and qualitative feedback. Network interruptions or API rate limits can introduce latency spikes.
- **Heuristic Document Parsing Variations:** PyMuPDF and regex boundary extraction assume standard resume layouts. Highly stylized, multi-column graphical resumes or non-standard font encodings occasionally misclassify technical skill boundaries, requiring manual profile corrections.
- **Absence of Visual Non-Verbal Analysis:** The current implementation focuses entirely on vocal and conceptual technical evaluation. Non-verbal signals (such as eye contact, posture, and facial anxiety cues) are not tracked, though studies indicate technical recruiters prioritize conceptual competence over visual cues in initial screening rounds.

IX. Conclusion and Future Work

This paper presented SmartInterview, a personalized, adaptive technical interview preparation system designed to address the placement preparation bottleneck in higher engineering education. By coupling Retrieval-Augmented Generation across a verified 402-chunk computer science corpus with a deterministic Bloom's Taxonomy state machine and a 5-factor scoring rubric, SmartInterview overcomes the hallucination, subjectivity, and lack of scaffolding endemic to standard conversational LLMs. Empirical benchmarks verified 100% Top-3 retrieval accuracy, sub-2.5 second inference latency, and high hallucination resistance across 33 domain benchmark queries. Systematic boundary tests (TC-01 through TC-07) confirmed complete functional stability across authentication, vector retrieval, adaptive state transitions, in-memory voice synthesis, and safety auto-stop mechanisms.

Future research directions include:

1. Executing the proposed longitudinal cohort study across B.Tech engineering students at KMIT during upcoming campus placement seasons.

2. Local quantization of open-source language models (e.g., Llama-3-8B-Instruct) via Ollama or vLLM to eliminate cloud API dependencies.

3. Bidirectional WebRTC full-duplex voice streaming for seamless real-time conversation.

References

1. L. W. Anderson and D. R. Krathwohl, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman, 2001.

2. B. S. Bloom, *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook 1: Cognitive Domain*. New York: David McKay Co., 1956.

3. P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459-9474, 2020.

4. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, pp. 3982-3992, 2019.

5. A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998-6008, 2017.

6. S. Wahid, M. A. Khan, and R. Ahmed, "Voice-driven automated mock interview simulator using neural language models," *Int. J. Emerg. Res. Technol. (IJERT)*, vol. 15, no. 2, pp. 112-118, Feb. 2026.

7. K. Nagarajan, P. Suresh, and V. Swaminathan, "Autonomous technical screening and resume skill alignment using dense semantic embeddings," *Int. J. Manag. IT Res. (IJMRR)*, vol. 16, no. 1, pp. 45-53, Jan. 2026.

8. J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, I. L. McClelland, and B. Weerdmeester, Eds. London: Taylor & Francis, 1996, pp. 189-194.

9. IEEE Standard for Systems and Software Engineering — Life Cycle Processes — Requirements Engineering, *IEEE Std 29148-2018*, Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.

10. T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877-1901, 2020.

11. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, pp. 4171-4186, 2019.

12. ChromaDB Development Team, "Chroma: The AI-native open-source vector database," 2024. [Online]. Available: https://docs.trychroma.com/

13. S. Tiangolo, "FastAPI: Modern, high-performance web framework for Python," 2024. [Online]. Available: https://fastapi.tiangolo.com/

14. M. McKerns et al., "PyMuPDF: High-performance rendering and document extraction for Python," 2024. [Online]. Available: https://pymupdf.readthedocs.io/

15. Groq Inc., "LPU Inference Engine and Architecture Overview," Whitepaper, Mountain View, CA, 2024. [Online]. Available: https://groq.com/

16. Oracle Corporation, "MySQL 8.0 Reference Manual: InnoDB Storage Engine and JSON Functions," 2024. [Online]. Available: https://dev.mysql.com/doc/

17. Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn. (ICML)*, pp. 41-48, 2009.

18. D. R. E. A. Bandura, "Self-efficacy: Toward a unifying theory of behavioral change," *Psychol. Rev.*, vol. 84, no. 2, pp. 191-215, 1977.

19. J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cogn. Sci.*, vol. 12, no. 2, pp. 257-285, 1988.

20. A. Radford et al., "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, pp. 28492-28518, 2023.

21. H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

22. M. Douze et al., "The Faiss library," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, 2024, doi: 10.1109/TPAMI.2024.3364234.

23. P. S. Dodds and D. J. Watts, "Universal behavior in a generalized model of contagion," *Phys. Rev. Lett.*, vol. 92, no. 21, p. 218701, 2004.

24. E. A. Locke and G. P. Latham, "Building a practically useful theory of goal setting and task motivation: A 35-year odyssey," *Am. Psychol.*, vol. 57, no. 9, pp. 705-717, 2002.

25. R. E. Mayer, *Multimedia Learning*, 3rd ed. Cambridge, U.K.: Cambridge Univ. Press, 2021.